ImageLab: Phase 1 Technical Report

Custom Dynamic Memory Representations, Core Complexities, and Pipeline Specification

Author: Ahmad Ali | **Roll Number:** 25-i0817 | **Section:** BS CS-E | **Course:** CS-2001 Data Structures

1. Architectural Overview and Design Philosophy

ImageLab Phase 1 implements a high-performance image processing engine and graphical studio engineered from scratch in C++17 strictly without forbidden language constructs: zero Standard Template Library (STL) containers (no vector, list, queue, stack, map), zero inline keywords, zero C++ enum constructs, and clean 1-function-per-header modular organization. Dynamic 2D matrices, Malik-compliant template data structures (stackType and circularQueueType), a dual-stack Undo/Redo transaction engine, a bounded circular processing pipeline, and an interactive SFML studio GUI establish a robust operational foundation.

2. Dynamic Memory Representations and Justifications (Section 6.8)

Image Memory Representation (Pixel Matrix):** Images are structured as an array of row pointers (Pixel**) with individual dynamic row allocations on the heap. This representation offers several critical advantages over 1D flattened buffers: (1) *Direct Cartesian Mapping:* Operations like 90-degree orthogonal rotations, cropping, and box blur convolutions index directly as img[y][x] without repeated stride multiplications (y * width + x) in tight inner loops; (2) *Zero-Copy Vertical Flips:* Vertical reflections swap horizontal row pointers in strictly O(H) time without re-allocating or moving image bytes; (3) *Fragmentation Resistance:* Allocating H independent row buffers succeeds reliably on fragmented heaps where a monolithic multi-megabyte 1D block would fail.

History Stacks (Bounded Array stackType): Follows the Malik Chapter 18 template specification. An array-based stack delivers O(1) push, pop, and top execution with zero pointer overhead per element, maximizing CPU L1 cache locality during rapid undo/redo cycles compared to a node-based linked list.

Processing Pipeline (Bounded Circular Array Queue): Implemented as a ring buffer with modular pointer advancement (rear = (rear + 1) % capacity). This guarantees strict O(1) enqueue and dequeue without O(N) memory shifts. *Full-Queue Policy:* When capacity (30 items) is reached, new tasks are rejected gracefully with a diagnostic status notification rather than crashing or overwriting unexecuted tasks. *Cancellation:* Handled in-place via an internal boolean flag (isCancelled = true), allowing safe removal without shifting queue elements.

3. Algorithmic Time and Space Complexity Analysis

Operation	Time Complexity	Aux Space	Engineering Mechanism and Invariants
Load / Save Image	O(W x H)	O(1)	Stream parsing with direct 2D row allocation.
Crop (x1, y1, x2, y2)	O(W' x H')	O(W' x H')	Allocates exact sub-matrix bounding box.
Rotate 90 deg / 270 deg	O(W x H)	O(W x H)	Dimension transposition matrix (H x W).
Rotate 180 deg	O(W x H)	O(1)	In-place double coordinate symmetry swap.
Horizontal Flip	O(W x H)	O(1)	In-place row element swapping.
Vertical Flip	O(H)	O(1)	In-place row pointer swapping without pixel moves.
Brightness / Grayscale	O(W x H)	O(1)	Direct pixel transformation clamped to [0, 255].
Bilinear Resize	O(W' x H')	O(W' x H')	Four-point interpolation for aliasing-free scaling.
Box Blur (3x3)	O(W x H)	O(W x H)	Frobenius inner product across 8-neighborhoods.
Bucket Fill (Stack DFS)	O(W x H)	O(W x H)	Bounded stack depth-first flood fill with visited mask.
Bucket Fill (Queue BFS)	O(W x H)	O(W x H)	Bounded circular queue breadth-first traversal.
Pipeline Enqueue / Dequeue	O(1)	O(1)	Modular ring-buffer index advancement.

4. Automated Test Evidence and Verification (Section 6.8)

The automated regression suite (tests/test.cpp) validates 100% of functional requirements: (1) **Stack and Queue Primitives:** Verified boundary checks, wrap-around index arithmetic, and exception handling; (2) **Transformation Invariants:** Proved Rotate_90^4(I) == I and Flip_H^2(I) == I; (3) **Flood Fill Parity:** Confirmed bit-for-bit identity between DFS and BFS bucket fills; (4) **Pipeline Capacity:** Validated graceful rejection on full capacity (30 jobs) and verified non-shifting cancellation; (5) **Dual Interface:** Both terminal console and interactive SFML GUI execute seamlessly.